

(Langchain ~~mai~~ Mai Jo bhi .invoke se hoga wo Runnable hai !!!)

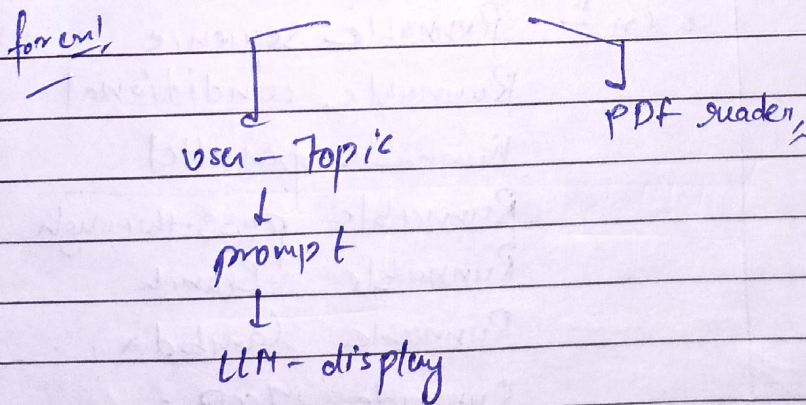
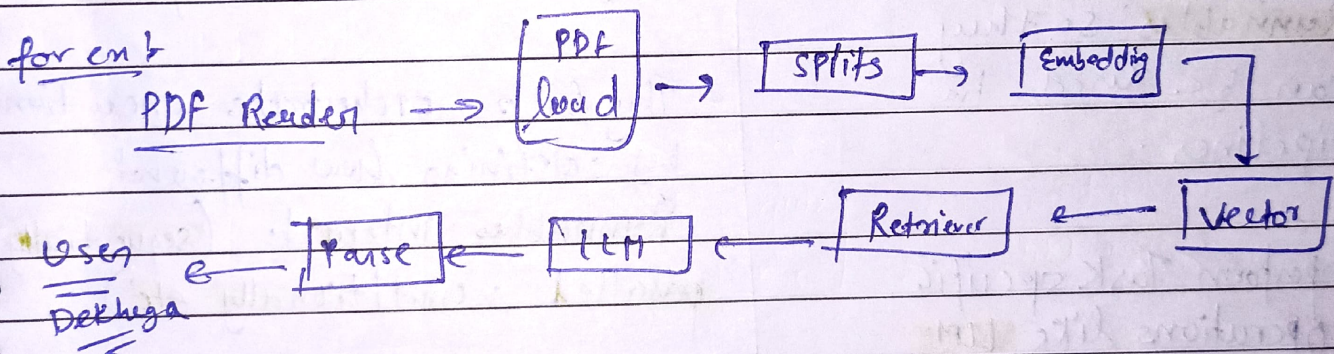
Date

Runnables :- ek executable unit Jo input leta hai process karta hai fir output deta hai.

2022 → chatgpt → Nov
↳ API → LLM based app

Langchain
↓
↳ chatgpt, PDF loaders, AI Agents
Anthropic, google.

framework
→ open AI
→ google
→ Anthropic.



→ Runnable is a standardized Interface in langchain that allows chaining models, prompts, parsers, tools retrievers, or custom logic into composable pipelines.

Input → Runnables → Output

(Long chain ke component like chat open AI, prompt template, intro Runnable Mai connect krte hai !!)

Runnables

(Task specific Runnables ko connect kr sktte hai !!)

Task specific
Runnables

Runnable
Primitive

- These are core langchain components that have been converted into Runnables so they can be used in pipelines.

- perform Task specific operations like LLM calls, prompting, retrieval etc.

Example :-

- chat open AI
- Prompt Template
- Retriever

- These are fundamental building block for structure execution logic in AI workflows

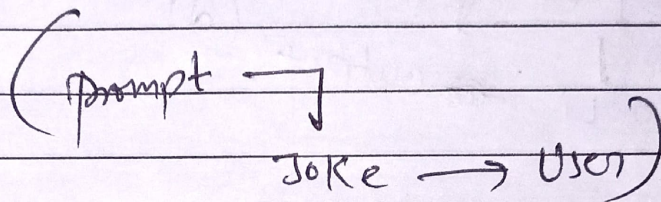
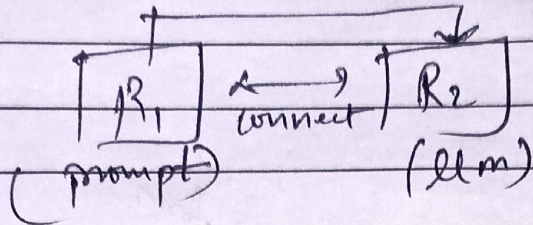
- They help orchestrate execution by defining how different Runnables interact (sequentially, parallel, conditionally etc).

- ex :- Runnable sequence
Runnable conditional
Runnable parallel
Runnable pass-through
Runnable Branch
Runnable lambda.
Runnable Map.

#1

Date

Runnable sequence :- (Do ya Do se Jayada Runnable Ko sequentially connect Krke Rkte hai !!)



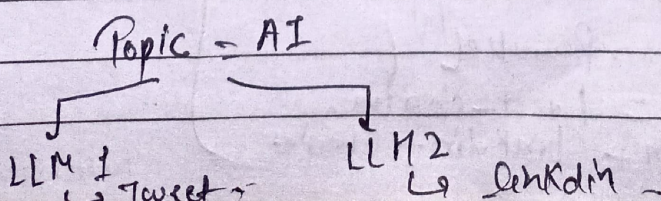
Runnable sequence.py → code on Github -

chain = prompt 1 | model | parser | prompt 2 | model | parser

result = chain.invoke({'topic': 'AI'})
print(result)

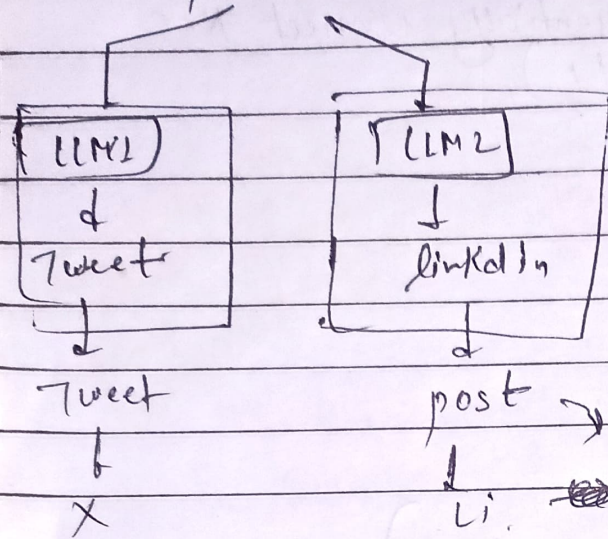
chain = RunnableSequence(prompt, model, parser)
print(chain.invoke({'topic': 'AI'}))

② Runnable Parallel :- (Runnable parallel apko parallel chains banane mai help Krta hai) !!

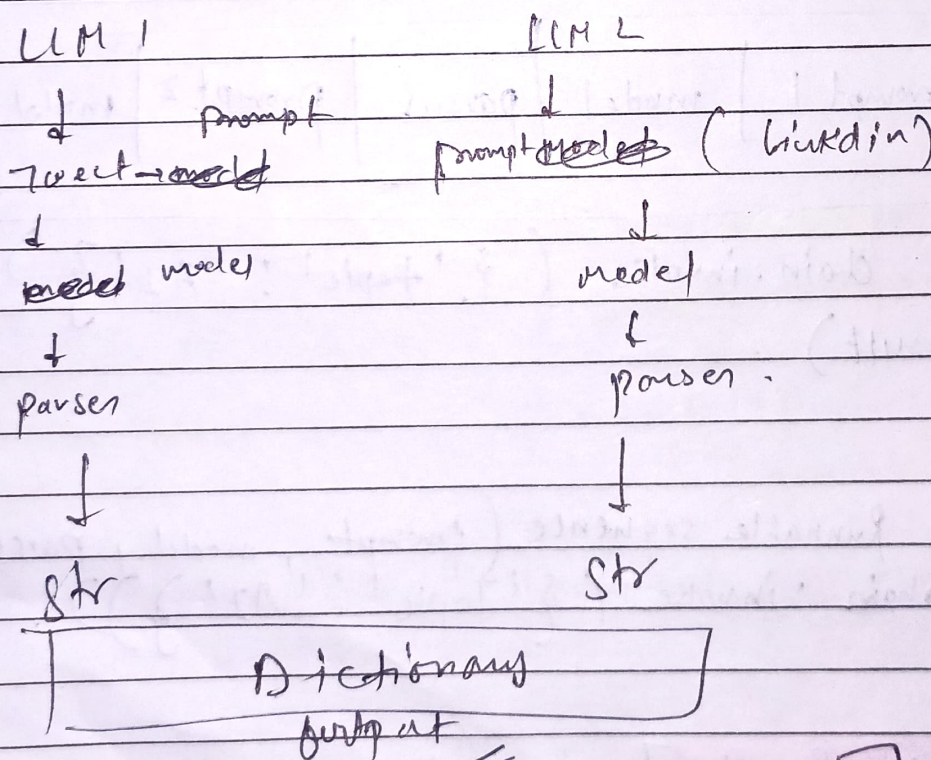


Teacher's Signature

Topic = AI → Input same



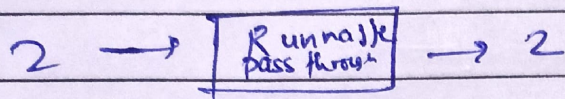
Runnable parallel.py $\frac{0}{5}$ codes on Github -



tweet chain = prompt 1 | model | parser
 LinkedIn chain = prompt 2 | model | parser
 Chain = RunnableParallel (
 tweet = tweet chain,
 LinkedIn = LinkedIn chain

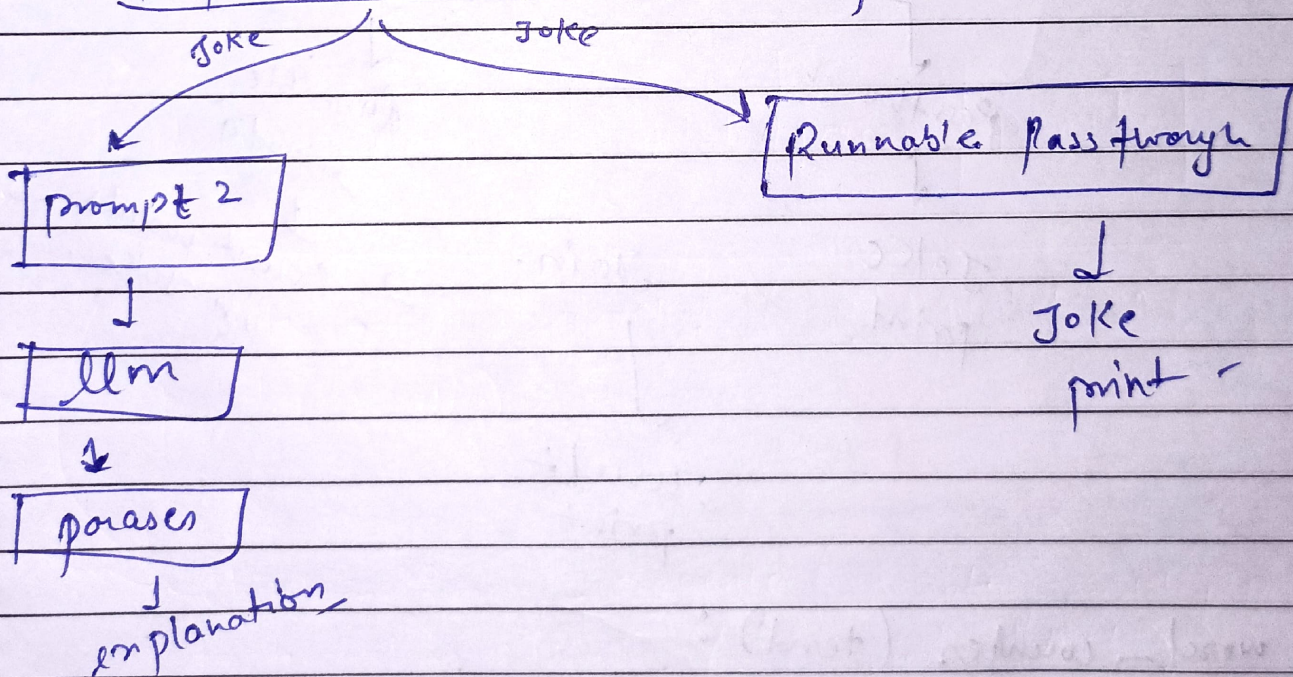
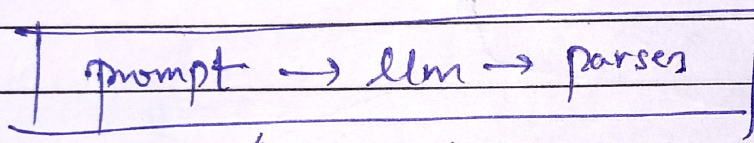
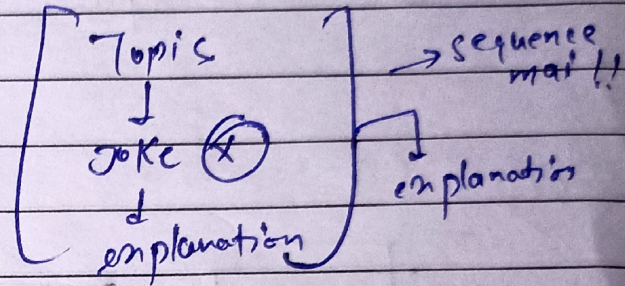
RunnableParallel //

(3) Runnable Pass through :- (ek special Runnable primitive hai jisko Jo input denge wahi palat ke output de dega !!) no modifying on Input :-



Joke → explanation
dono print krna hai.

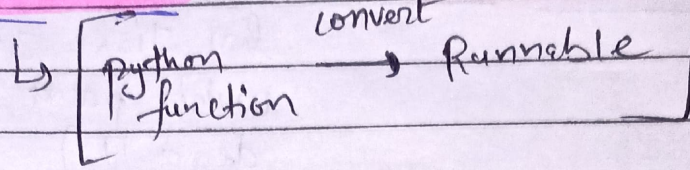
Joke-generator-chain



(4)

Runnable lambda's

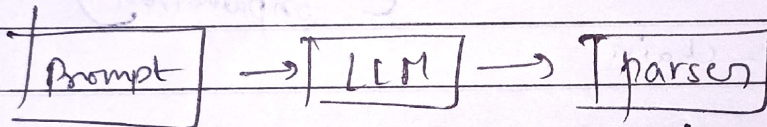
code on gitlab



Joke → Topic → _____

(number of word
kitha hai)

exit



pass through

joke
print

Join.

result
printRunnable
lambdacount words
in joke.

```

def word_counter(text):
    return len(text.split())
  
```

x

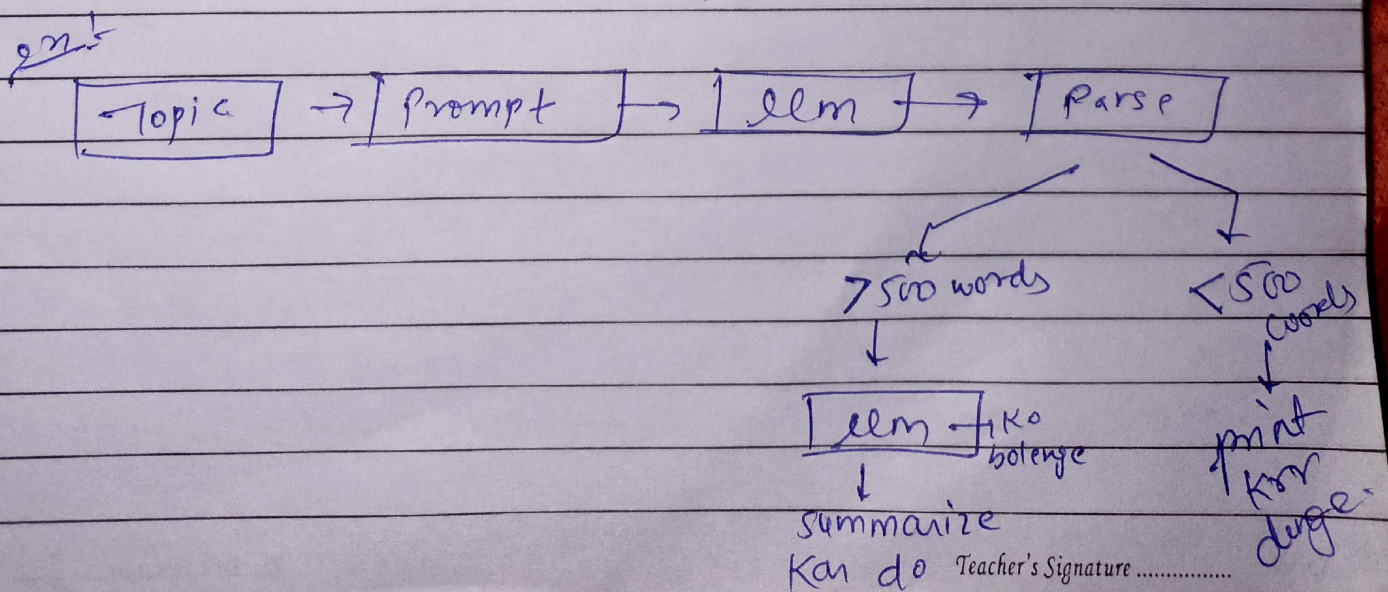
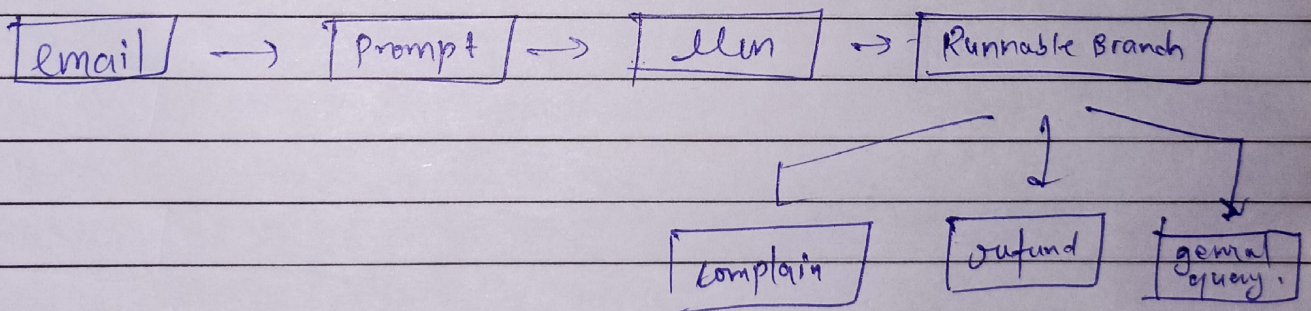
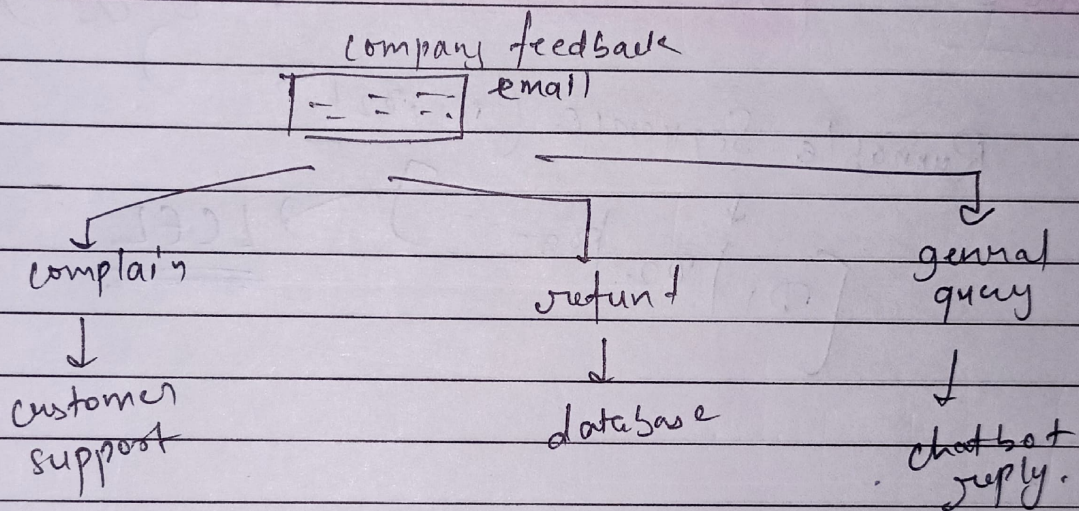
x

x

x

(5) Runnable Branch ÷ Conditional chain banana ke liye.

↳ it function like an if/elif/else blocks for chains
 - where you define a set of condition functions, each associated with a runnable (eg. LLM prompt chain, or tool). the first matching call is executed if no condition matches, a default runnable is used.



Runnable Branch (

($\rightarrow$ if condition

($\rightarrow$ if condition

$\rightarrow$ default condition

)

($\rightarrow$, runnable)

LC EL $\rightarrow$ (langchain Expression language)

Runnable Sequence ($r_1, r_2, r_3, \dots$)

$\downarrow$
 $[r_1 | r_2 | r_3 \dots] \rightarrow$ LC EL